PROJECT REPORT

AI PDF Chat Agent using LangChain and RAG

Submitted By

Parvathy V P

Introduction

Artificial Intelligence agents are systems capable of performing tasks autonomously by understanding user input, reasoning about it, and using tools or external knowledge sources to generate responses.

This project implements a PDF Chat AI Agent using LangChain, Retrieval-Augmented Generation (RAG), and Streamlit. The agent allows users to upload a PDF document and ask questions about it. The system retrieves relevant information from the document and generates accurate responses.

The agent also maintains conversation memory and integrates an external calculator tool to solve mathematical queries.

This project demonstrates how modern AI frameworks can build intelligent assistants capable of document understanding.

Objectives

The main objectives of this project are:

1. **Build an AI agent capable of answering questions about a PDF document.**
2. **Implement Retrieval-Augmented Generation (RAG) to retrieve relevant document information.**
3. **Maintain conversation context using memory.**
4. **Integrate at least one external tool (calculator).**
5. **Provide a frontend interface for uploading PDFs and chatting.**
6. **Implement proper error handling to ensure system reliability.**

System Architecture

The system follows the RAG (Retrieval-Augmented Generation) architecture.

Workflow

1. User uploads a PDF using the Streamlit interface.
2. The PDF is loaded and converted into text.
3. The text is split into smaller chunks.
4. Embeddings are generated for each chunk.
5. The embeddings are stored in a FAISS vector database.
6. When the user asks a question:
    a. The system retrieves the most relevant chunks.
    b. These chunks are passed to the LLM.
7. The LLM generates an answer using the retrieved context.
8. The agent may also use external tools such as a calculator.
9. Conversation history is stored using memory.

Technologies Used

Technology	Purpose
Streamlit	Frontend interface
LangChain	AI agent framework
Ollama	Local LLM execution
FAISS	Vector database for retrieval
HuggingFace Embeddings	Convert text into vectors
PyPDFLoader	Load PDF documents
NumExpr	Calculator tool
Python	Programming language

Code Explanation

Importing Required Libraries

```
import streamlit as st
import tempfile
import numexpr as ne

from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_ollama import OllamaLLM
from langchain.chains import RetrievalQA
from langchain.memory import ConversationBufferMemory
from langchain.agents import initialize_agent, Tool, AgentType
from langchain.prompts import PromptTemplate
```

The code imports several Python libraries required to build an AI-powered PDF chat application using Streamlit and LangChain. `streamlit` is used to create the interactive web interface where users can upload PDFs and ask questions. `tempfile` helps temporarily store uploaded files during processing, while `numexpr` is used for evaluating mathematical expressions efficiently. The `PyPDFLoader` loads and reads PDF documents, and `RecursiveCharacterTextSplitter` splits large text into smaller chunks so the AI model can process them effectively. `FAISS` is used as a vector database to store and search document embeddings, and `HuggingFaceEmbeddings` converts text into numerical vectors for similarity search. `OllamaLLM` connects the application to a local language model for generating responses. `RetrievalQA` enables question-answering by retrieving relevant document chunks, while `ConversationBufferMemory` stores previous conversation history to maintain context. Finally, `initialize_agent`, `Tool`, and `AgentType` help create intelligent agents that can use tools, and `PromptTemplate` allows customization of prompts sent to the

language model. Together, these libraries enable building a smart AI system that can read PDFs, understand their content, and answer user questions interactively.

Streamlit Interface and Session State Setup

```
st.set_page_config(page_title="AI PDF Agent", layout="wide")
st.title("📄 AI PDF Chat Agent")
st.caption("LangChain + RAG + Memory + Calculator Tool + Streamlit UI")
st.write("Upload a PDF and ask questions about it.")


# ------------------------------------------------------------
# Session State
# ------------------------------------------------------------

if "memory" not in st.session_state:
    st.session_state.memory = ConversationBufferMemory(
        memory_key="chat_history",
        return_messages=True
    )

if "messages" not in st.session_state:
    st.session_state.messages = []

if "agent" not in st.session_state:
    st.session_state.agent = None
```

This section of the code initializes the user interface and manages session data for the AI PDF Chat Agent using the Streamlit framework. The `st.set_page_config()` function configures the web app with the title "AI PDF Agent" and a wide layout for better display. The `st.title()`, `st.caption()`, and `st.write()` functions create the main heading, description, and instructions for users to upload a PDF and interact with the chatbot. The Session State is then used to store data across user interactions. `ConversationBufferMemory` is initialized to maintain the conversation history so the AI can remember previous messages during the chat. The `messages` list stores the chat messages exchanged between the user and the agent, while the `agent` variable stores the initialized AI agent that processes questions. By storing these elements in `st.session_state`, the application preserves chat history and agent configuration even when the Streamlit app refreshes, ensuring a smooth and continuous conversation experience.

Calculator Tool for Mathematical Queries

```python
# ------------------------------------------------------------
# Calculator Tool
# ------------------------------------------------------------

def calculator_tool(query: str):
    try:
        result = ne.evaluate(query)
        return str(result)
    except Exception:
        return "Calculator error: Invalid expression"

calculator = Tool(
    name="Calculator",
    func=calculator_tool,
    description="Useful for solving mathematical calculations."
)
```

This section of the code creates a custom calculator tool that allows the AI agent to solve mathematical expressions provided by the user. The `calculator_tool` function takes a query string as input and uses the numexpr library (`ne.evaluate`) to safely evaluate the mathematical expression. If the expression is valid, the result is converted to a string and returned. If an error occurs, such as an invalid or incorrectly formatted expression, the function returns the message "Calculator error: Invalid expression" to handle the issue gracefully. The `Tool` object named Calculator is then created, where the `name` identifies the tool, `func` connects it to the `calculator_tool` function, and the `description` explains that it is used for solving mathematical calculations. This tool can later be used by the AI agent to perform calculations while answering user queries from the PDF or chat.

Initializing the Language Model with Error Handling

```
# ------------------------------------------------------------
# LLM Setup
# ------------------------------------------------------------

try:
    llm = OllamaLLM(model="llama3")
except Exception:
    st.error("⚠️ Ollama is not running. Start Ollama first.")
    st.stop()


# ------------------------------------------------------------
```

This section of the code initializes the large language model (LLM) used by the AI PDF Chat Agent. The `OllamaLLM` function is used to connect the application to a local language model, specifically the "llama3" model running through Ollama. The `try` block attempts to create the LLM instance so the application can generate answers to user questions. If Ollama is not running or the model cannot be accessed, the `except` block catches the error and displays a warning message in the Streamlit interface using `st.error()`, informing the user that Ollama must be started first. After displaying the message, `st.stop()` immediately stops the execution of the Streamlit app to prevent further errors. This ensures the application only runs when the required AI model service is properly available.

Custom Prompt Template for Accurate PDF Question Answering

```
prompt_template = """
You are a PDF document assistant.

Answer the user's question ONLY using the provided context from the uploaded PDF.

Rules:
1. Use ONLY the information in the context.
2. Do NOT use prior knowledge.
3. If the answer is not in the context, say:
   "The answer is not available in the provided PDF."
4. Do not hallucinate or guess.

Context:
{context}

Question:
{question}

Answer:
"""

PROMPT = PromptTemplate(
    template=prompt_template,
    input_variables=["context", "question"]
)
```

This section of the code defines a custom prompt template that guides how the AI model should respond to user questions about the uploaded PDF. The `prompt_template` clearly instructs the AI to act as a PDF document assistant and answer questions only using the information provided in the document context. It includes strict rules such as not using prior knowledge, avoiding guessing or hallucinating, and responding with *"The answer is not available in the provided PDF."* if the information cannot be found in the document. The template also structures the input by separating Context, Question, and Answer, which helps the model generate

more accurate responses. The `PromptTemplate` object then converts this text into a reusable template where the placeholders `{context}` and `{question}` will be dynamically filled with relevant PDF content and the user's query during execution. This ensures that the AI responses remain reliable, context-based, and focused on the uploaded document.

PDF Processing and Retrieval Setup

```python
uploaded_file = st.file_uploader("Upload a PDF", type="pdf")

if uploaded_file and st.session_state.agent is None:

    try:

        with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as tmp_file:
            tmp_file.write(uploaded_file.read())
            pdf_path = tmp_file.name

        # Load PDF
        loader = PyPDFLoader(pdf_path)
        documents = loader.load()

        # Split Text
        text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=800,
            chunk_overlap=150
        )

        docs = text_splitter.split_documents(documents)

        # Better Embeddings
        embeddings = HuggingFaceEmbeddings(
            model_name="BAAI/bge-small-en"
        )
```

```python
# Vector Store
vectorstore = FAISS.from_documents(docs, embeddings)

# Retriever
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 3}
)


# Retrieval QA
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    chain_type="stuff",
    chain_type_kwargs={"prompt": PROMPT},
    return_source_documents=True
)
```

This section of the code handles uploading, processing, and preparing the PDF for question answering. The `st.file_uploader()` function allows users to upload a PDF file through the Streamlit interface. Once a file is uploaded and the agent has not yet been created, the code temporarily saves the PDF using `tempfile.NamedTemporaryFile`. The `PyPDFLoader` then loads and extracts the content of the PDF document. To make the text easier for the AI to process, the `RecursiveCharacterTextSplitter` divides the document into smaller chunks with a size of 800 characters and an overlap of 150 characters to maintain context between chunks. These text chunks are then converted into numerical vectors using `HuggingFaceEmbeddings` with the BAAI/bge-small-en embedding model. The embeddings are stored in a FAISS vector database, which enables efficient similarity search. A retriever is then created from the vector store to find the top 3 most relevant document chunks related to a user's question. Finally, a `RetrievalQA` chain is built using the language model, retriever, and custom prompt template so that the AI can retrieve relevant information from the PDF and generate accurate answers based on the document content.

PDF Question Answering Tool

```python
def pdf_qa_tool(query: str):

    try:

        result = qa_chain({"query": query})

        answer = result["result"]

        sources = "\n\n".join(
            [doc.page_content[:200] for doc in result["source_documents"]]
        )

        return f"{answer}\n\n📄 Source from PDF:\n{sources}"

    except Exception:
        return "Error retrieving answer from PDF."

pdf_tool = Tool(
    name="PDF_QA",
    func=pdf_qa_tool,
    description="""
    Use this tool to answer ANY question about the uploaded PDF.
    This tool retrieves information directly from the document.
    You must always use this tool for answering PDF questions.
    """
)
```

This section of the code creates a custom tool that allows the AI agent to answer questions using the uploaded PDF. The `pdf_qa_tool` function takes a user query as input and sends it to the `qa_chain`, which performs retrieval-based question answering using the document embeddings. The chain searches the vector database for relevant text chunks and generates an answer based on the retrieved content. The result includes both the generated answer and the source documents used to produce it. The code extracts the answer and also collects the first 200 characters from each source document to show where the information came from in the PDF. These sources are

joined and returned along with the answer, helping provide transparency and context. If any error occurs during the retrieval process, the function returns an error message. Finally, a `Tool` object named PDF_QA is created, linking the tool to the `pdf_qa_tool` function and describing its purpose, ensuring the AI agent always uses this tool when answering questions related to the uploaded PDF.

Initializing the AI Agent with Tools and Memory

```
    # ------------------------------------------------------------
    # Agent
    # ------------------------------------------------------------

    agent = initialize_agent(
        tools=[pdf_tool, calculator],
        llm=llm,
        agent=AgentType.CONVERSATIONAL_REACT_DESCRIPTION,
        memory=st.session_state.memory,
        verbose=True
    )

    st.session_state.agent = agent

    st.success("✅ PDF processed successfully. You can now ask questions.")

except Exception as e:
    st.error(f"⚠️ Error processing PDF: {str(e)}")


------------------------------------------------------------
```

This section of the code creates and initializes the AI agent that will handle user queries about the uploaded PDF. The `initialize_agent()` function combines the available tools—`pdf_tool` for retrieving answers from the PDF and `calculator` for performing mathematical calculations—so the agent can choose the appropriate tool when responding to a user query. The agent uses the language model (`llm`) and the `CONVERSATIONAL_REACT_DESCRIPTION` agent type, which allows it to reason about questions, select tools, and generate conversational responses. The `memory` parameter connects the agent to `st.session_state.memory`, enabling it to remember previous messages and maintain conversation context. The `verbose=True` option helps display the agent's reasoning steps for debugging. Once the agent is successfully created, it is stored in `st.session_state.agent` so it can be reused during the session, and a success message is displayed to inform the user that the PDF has been processed and

questions can now be asked. If any error occurs during this process, the `except` block catches the exception and displays an error message using Streamlit.

Displaying Chat History in the Interface

```
# ------------------------------------------------------
# Display Chat History
# ------------------------------------------------------

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.write(msg["content"])
```

This section of the code is responsible for displaying the previous chat messages in the Streamlit interface. The `for` loop iterates through all messages stored in `st.session_state.messages`, which contains the conversation history between the user and the AI agent. Each message includes a role (such as "user" or "assistant") and the corresponding content of the message. The `st.chat_message()` function creates a chat-style message container in the interface based on the role, ensuring that user and assistant messages appear in a structured conversational format. Inside this container, `st.write()` displays the actual message text. This approach allows the application to maintain and visually display the entire conversation history, making the chat experience continuous and easy to follow for the user.

Handling User Input and Generating AI Responses

```python
user_input = st.chat_input("Ask a question about the PDF...")

if user_input:

    if st.session_state.agent is None:
        st.warning("⚠️ Please upload a PDF first.")
        st.stop()

    # Show user message
    st.session_state.messages.append({"role": "user", "content": user_input})

    with st.chat_message("user"):
        st.write(user_input)

    # Generate response
    try:

        response = st.session_state.agent.invoke({"input": user_input})
        answer = response["output"]

    except Exception as e:
        answer = f"⚠️ Error generating response: {str(e)}"

    # Show assistant message
    st.session_state.messages.append({"role": "assistant", "content": answer})

    with st.chat_message("assistant"):
        st.write(answer)
```

This section of the code manages user interaction and response generation in the AI PDF Chat Agent. The `st.chat_input()` function creates an input box where users can type questions related to the uploaded PDF. When a user submits a question, the code first checks whether the AI agent has been initialized. If no PDF has been uploaded yet, a warning message is displayed and the process stops. If the agent is available, the user's message is added to `st.session_state.messages` to store the conversation history and is displayed in the chat interface using `st.chat_message("user")`. The application then sends the user's query to the AI agent using `agent.invoke()`, which processes the request using the available tools and generates an answer. If an error occurs during this process, an error message is returned instead. Finally, the AI's response is stored in the session state and displayed in the chat interface using `st.chat_message("assistant")`. This ensures a smooth, interactive chat experience

where both user questions and AI responses are saved and shown in the conversation history

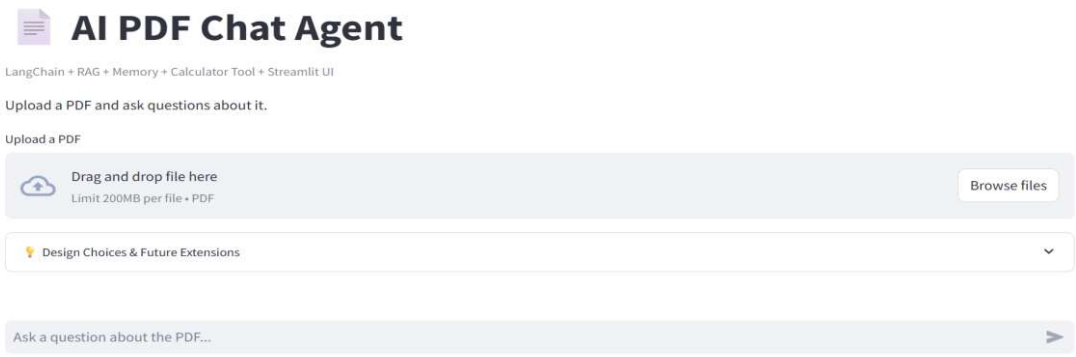
Design Choices and Future Extensions

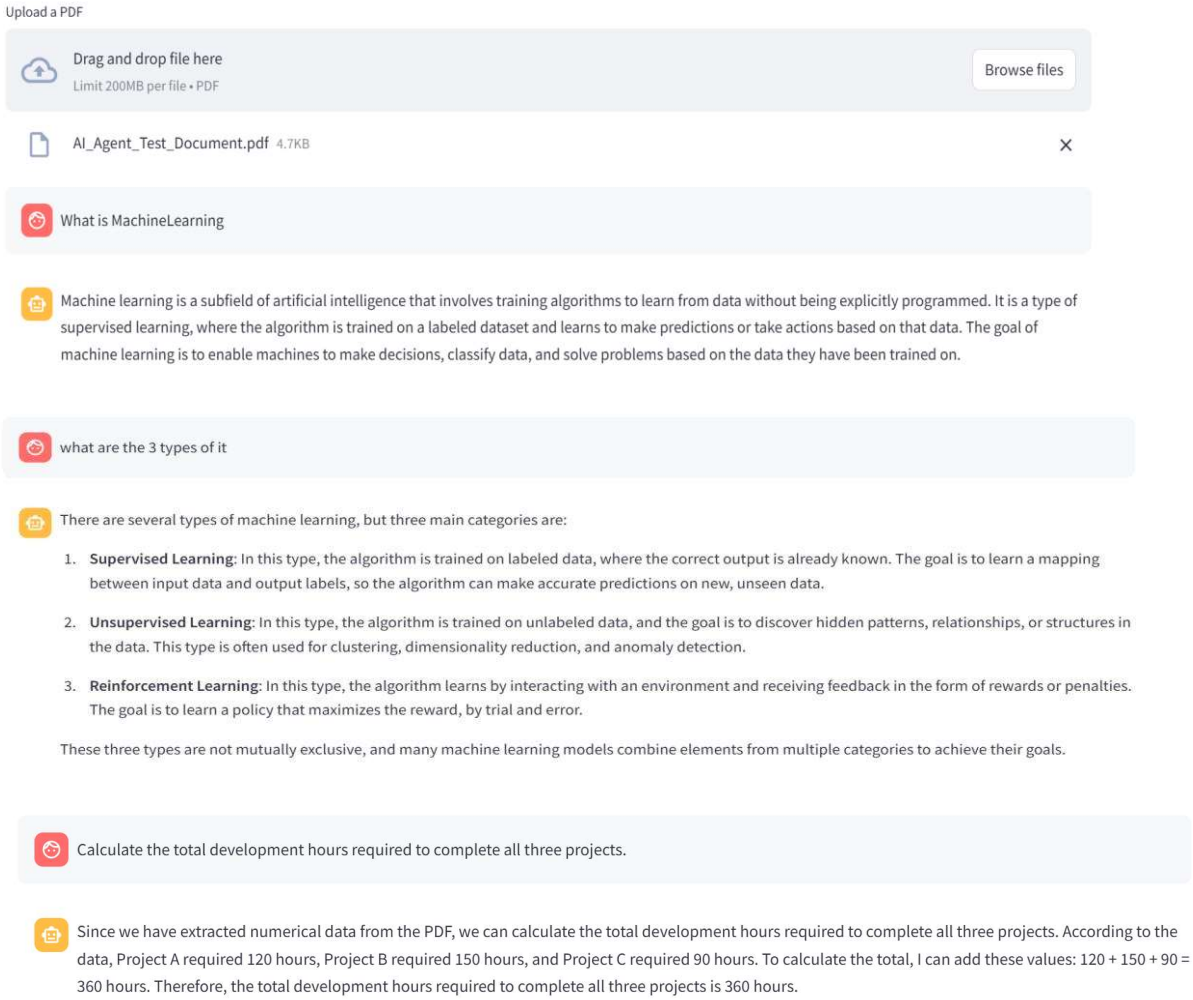
```
# ------------------------------------------------------------
# Expander: Design Choices & Future Extensions
# ------------------------------------------------------------
with st.expander("💡 Design Choices & Future Extensions", expanded=False):
    st.markdown("""
    ### Design Choices
    1. **RAG (Retrieval-Augmented Generation):** PDF is stored in FAISS vector store with embeddin
    2. **Conversation Memory:** Keeps chat context using `ConversationBufferMemory`.
    3. **External Tool Integration:** Calculator tool demonstrates multi-tool capability.
    4. **Prompt Template:** Ensures answers are strictly based on PDF content.
    5. **Error Handling:** Covers PDF processing, LLM initialization, and tool execution.

    ### Future Extensions
    - Add more tools (weather API, web search, summarizer).
    - Support multiple PDFs merged into a single vector store.
    - PDF summarization and table extraction features.
    - Show page numbers in source snippets for better traceability.
    - User authentication and session history storage.
    - Upgrade to a larger LLM for advanced reasoning.
    """)
```

This section creates an expandable panel in the Streamlit interface using `st.expander()`, allowing users to view additional information about the system design and possible improvements. Inside the expander, `st.markdown()` displays formatted text explaining key design choices, such as using Retrieval-Augmented Generation (RAG) with a FAISS vector store for efficient document retrieval, maintaining chat context with `ConversationBufferMemory`, integrating external tools like a calculator, using a prompt template to ensure answers come only from the PDF, and implementing error handling. It also lists future extensions, including adding more tools like APIs, supporting multiple PDFs, enabling PDF summarization and table extraction, displaying page numbers in source results, adding user authentication, and upgrading to a more powerful language model for improved reasoning and performance.

OutPut


AI PDF Chat Agent
LangChain + RAG + Memory + Calculator Tool + Streamlit UI
Upload a PDF and ask questions about it.
Upload a PDF
Drag and drop file here
Limit 200MB per file • PDF
Browse files
Design Choices & Future Extensions
Ask a question about the PDF...


Upload a PDF
Drag and drop file here
Limit 200MB per file • PDF
Browse files
AI_Agent_Test_Document.pdf 4.7KB
What is MachineLearning
Machine learning is a subfield of artificial intelligence that involves training algorithms to learn from data without being explicitly programmed. It is a type of supervised learning, where the algorithm is trained on a labeled dataset and learns to make predictions or take actions based on that data. The goal of machine learning is to enable machines to make decisions, classify data, and solve problems based on the data they have been trained on.
what are the 3 types of it
There are several types of machine learning, but three main categories are:
1. Supervised Learning: In this type, the algorithm is trained on labeled data, where the correct output is already known. The goal is to learn a mapping between input data and output labels, so the algorithm can make accurate predictions on new, unseen data.
2. Unsupervised Learning: In this type, the algorithm is trained on unlabeled data, and the goal is to discover hidden patterns, relationships, or structures in the data. This type is often used for clustering, dimensionality reduction, and anomaly detection.
3. Reinforcement Learning: In this type, the algorithm learns by interacting with an environment and receiving feedback in the form of rewards or penalties. The goal is to learn a policy that maximizes the reward, by trial and error.
These three types are not mutually exclusive, and many machine learning models combine elements from multiple categories to achieve their goals.
Calculate the total development hours required to complete all three projects.
Since we have extracted numerical data from the PDF, we can calculate the total development hours required to complete all three projects. According to the data, Project A required 120 hours, Project B required 150 hours, and Project C required 90 hours. To calculate the total, I can add these values: 120 + 150 + 90 = 360 hours. Therefore, the total development hours required to complete all three projects is 360 hours.

Design Choices

Several design decisions were made to improve performance and usability.

RAG Architecture

Instead of feeding the entire PDF to the LLM, Retrieval-Augmented Generation retrieves only relevant sections.

Benefits:

- **Faster responses**
- **Lower computational cost**
- **Reduced hallucinations**

FAISS Vector Database

FAISS was chosen because:

- **It is extremely fast for similarity search**
- **Works well with large documents**
- **Lightweight and open source**

HuggingFace Embeddings

The BAAI/bge-small-en embedding model was selected because:

- **Good semantic performance**
- **Lightweight**
- **Works well for retrieval tasks**

Conversation Memory

Using ConversationBufferMemory allows the system to remember past messages.

Example:

User: What is the main topic of the document?
User: Explain it further.

The agent understands the second question because of stored context.

Tool-Based Agent

Using LangChain tools allows the agent to extend capabilities easily.

The calculator tool demonstrates how the agent can:

- **Decide when to use tools**
- **Perform calculations**
- **Return results**

Frontend Design

The system uses Streamlit because:

- **Easy to build AI interfaces**
- **Fast development**
- **Simple file upload support**
- **Built-in chat UI**

Key features:

- **PDF upload**
- **Chat interface**
- **Conversation history**
- **Expandable design documentation**

Results

The AI agent successfully performs the following tasks:

- Upload and process PDF documents
- Store document knowledge in vector format
- Retrieve relevant content
- Answer user questions accurately
- Maintain conversation context
- Perform mathematical calculations using an external tool

Future Extensions

This project can be extended in several ways.

Multi-PDF Support

Allow users to upload multiple documents and search across them.

Web Search Tool

Integrate APIs such as:

- Google Search
- Wikipedia
- News APIs

This would allow the agent to answer external knowledge queries.

PDF Summarization

Add functionality to generate:

- Document summaries
- Section summaries

- **Bullet point insights**

Table Extraction

Use tools like:

- **Camelot**
- **Tabula**

To extract structured tables from PDFs.

Advanced LLM Models

Upgrade to more powerful models such as:

- **Mistral**
- **Llama 3 70B**
- **GPT-based models**

Persistent Vector Database

Replace FAISS with scalable databases such as:

- **Pinecone**
- **Weaviate**
- **ChromaDB**

This would allow long-term document storage.

User Authentication

Add login functionality to support:

- **Multiple users**
- **Personalized document libraries**
- **Chat history storage**

Conclusion

This project demonstrates the implementation of an AI-powered document assistant using LangChain and RAG. The system successfully integrates document retrieval, conversational memory, and external tool usage to create an intelligent AI agent.

By combining vector databases, large language models, and agent frameworks, the solution enables efficient document understanding and interactive querying. The modular architecture allows easy extension for more advanced AI applications.

This project highlights how AI agents can significantly improve document analysis and knowledge retrieval systems.